

REMEDIATION GUIDE

Web Application Penetration Testing — Project 3

Target: Damn Vulnerable Web Application (DVWA)

Assessment Environment: Authorized Local Lab

Prepared By: Aliza Javed

Date: 31 August 2026

1. SQL Injection — High

Issue

User input is directly affecting SQL query execution, allowing an attacker to manipulate database queries.

Remediation

- Use **prepared statements / parameterized queries**.
- Never concatenate user input directly into SQL queries.
- Validate input according to the expected data type.
- Use least-privilege database accounts.
- Disable detailed database error messages in production.

Secure Practice

Treat all user-controlled input as untrusted data.

2. Reflected XSS — Medium

Issue

User-supplied input is reflected in the web response without proper output encoding, allowing JavaScript execution in the victim's browser.

Remediation

- Apply **context-aware output encoding**.
- Validate user input on the server side.
- Avoid unsafe HTML/DOM manipulation.
- Sanitize HTML where user-supplied HTML is legitimately required.
- Implement an appropriate **Content Security Policy (CSP)**.

Secure Practice

Encode untrusted data before displaying it in an HTML, JavaScript, URL, or attribute context.

3. Stored XSS — High

Issue

Malicious user input can be stored by the application and executed when the stored content is viewed.

Remediation

- Encode stored user-generated content before displaying it.
- Use trusted HTML sanitization when HTML input is required.
- Restrict who can create persistent content.
- Avoid unsafe DOM APIs.
- Implement Content Security Policy.

Secure Practice

Never treat database-stored user content as trusted simply because it has already been stored.

4. Cross-Site Request Forgery (CSRF) — Medium

Issue

A state-changing password request can be processed without an adequate server-validated CSRF protection mechanism.

Remediation

- Implement unpredictable **CSRF tokens**.

- Bind tokens to the user's session.
- Validate tokens on the server for every sensitive state-changing request.
- Use **POST/PUT/PATCH** instead of GET for state-changing operations.
- Configure appropriate **SameSite** cookie protection.

Secure Practice

Sensitive actions such as password changes, account modifications, and financial operations must require verified user intent.

5. Authentication Security

Recommended Hardening

- Enforce strong password policies.
 - Store passwords using secure password-hashing algorithms.
 - Implement login rate limiting/throttling.
 - Protect authentication forms against CSRF.
 - Consider MFA for privileged accounts.
 - Monitor repeated failed login attempts.
-

6. Session Security

Recommended Hardening

- Use **HttpOnly** for session cookies.
 - Use **Secure** when HTTPS is enabled.
 - Configure an appropriate **SameSite** policy.
 - Regenerate the session ID after successful authentication.
 - Implement session expiration and idle timeouts.
 - Invalidate sessions properly during logout.
 - Use HTTPS for authenticated communication.
-

7. File Upload Security

Recommended Hardening

- Allow only required file types.
 - Validate both file extension and file content.
 - Apply file-size restrictions.
 - Generate safe server-side filenames.
 - Store uploaded files outside the web root where possible.
 - Prevent uploaded files from being executed as server-side scripts.
 - Apply malware scanning where appropriate.
-

8. General Security Recommendations

The following controls should be applied across the application:

- Validate all user input.
 - Treat all external data as untrusted.
 - Use secure authentication and authorization.
 - Apply the principle of least privilege.
 - Keep frameworks and dependencies updated.
 - Use HTTPS.
 - Implement appropriate security headers.
 - Log security-relevant events without storing passwords or session tokens.
 - Perform regular vulnerability assessments and penetration testing.
-

9. Remediation Verification

After applying the recommended fixes, perform a security retest to confirm:

- SQL Injection payloads no longer alter database queries.
- XSS payloads are rendered as text rather than executed.
- CSRF-protected requests fail when the token is missing or invalid.
- Authentication controls reject unauthorized access attempts.
- Session cookies use appropriate security attributes.
- Uploaded files cannot be executed as server-side scripts.